第4周：从OOA到OOP

动态建模与架构设计的综合运用

面向对象分析与设计实践

授课教师：_____

时间：2学时

前置知识：第3讲 OOAD、第4讲 UML规范

回顾与引入

第3周我们做了什么?

- **OOA**：从需求中提取名词，发现对象（User、Book、BorrowRecord）
- **OOD**：设计类结构，应用SOLID原则，绘制类图
- **OOP**：用Rust实现基础代码（Student、Book、BorrowRecord）

本周的问题

- 有了静态结构（类图），如何描述**动态行为**?
- 系统如何与用户交互？对象之间如何协作?
- 对象内部状态如何变化？业务流程如何流转?
- 系统如何部署？组件如何组织?

答案：动态建模 + 架构设计

类型	描述
动态建模（顺序图、状态图、活动图）	描述系统的**行为**
架构设计（组件图、部署图）	描述系统的**物理结构**

最终通过**迭代细化**，将设计转化为可运行的OOP代码

OOA → OOD → OOP 的核心思想

OOA → OOD → OOP 的核心思想

OOA（面向对象分析）

“做什么”

- 理解问题域，识别现实世界中的对象、属性、关系
- **产出**：用例图、对象发现列表、领域模型

OOD（面向对象设计）

“怎么做”

- 定义软件对象、接口、协作，应用设计原则与模式
- **产出**：类图、顺序图、状态图、活动图、组件图、部署图

OOP（面向对象编程）

“实现”

- 用具体编程语言实现设计，将对象转化为代码
- **产出**：可运行的软件系统

本周定位：在OOD层面完善动态模型与架构，并迭代细化至OOP

UML图例的综合运用

为什么要这么多图?

UML图例的综合运用——为什么要这么多图?

UML是一套”多视角”建模语言

UML图	问题	视角
用例图	谁用系统？做什么？	需求视角
类图	系统有哪些对象？它们有什么关系？	静态结构视角

UML图	问题	视角
顺序图	对象如何协作完成一个用例?	交互视角
状态图	单个对象在生命周期中如何变化?	状态视角
活动图	业务流程如何流转?	过程视角
组件图	系统由哪些可替换的物理部件组成?	模块视角
部署图	软件运行在什么硬件上?	物理视角

综合运用的意义

- 不同图回答不同问题，**互为补充**
- 从不同角度验证设计的**一致性与完整性**
- 便于**团队沟通**，降低理解偏差
- 为代码实现提供**清晰的蓝图**

以”借书”为例

如何综合运用UML图

以”借书”为例——如何综合运用UML图

场景：学生借一本书

UML图	作用
用例图	明确”借书”是学生的用例，与”查询图书”有关联
类图	定义参与借书的类（User、Book、BorrowRecord、BorrowService）
顺序图	描述学生、界面、控制器、服务、数据库之间的消息交互
状态图	描述图书从”可借”到”已借出”的状态变化
活动图	描述借书流程中的决策、并行活动（如检查权限、更新库存）
组件图	将借书功能封装为服务组件，明确依赖

UML图	作用
部署图	决定借书服务部署在哪个服务器，客户端如何访问

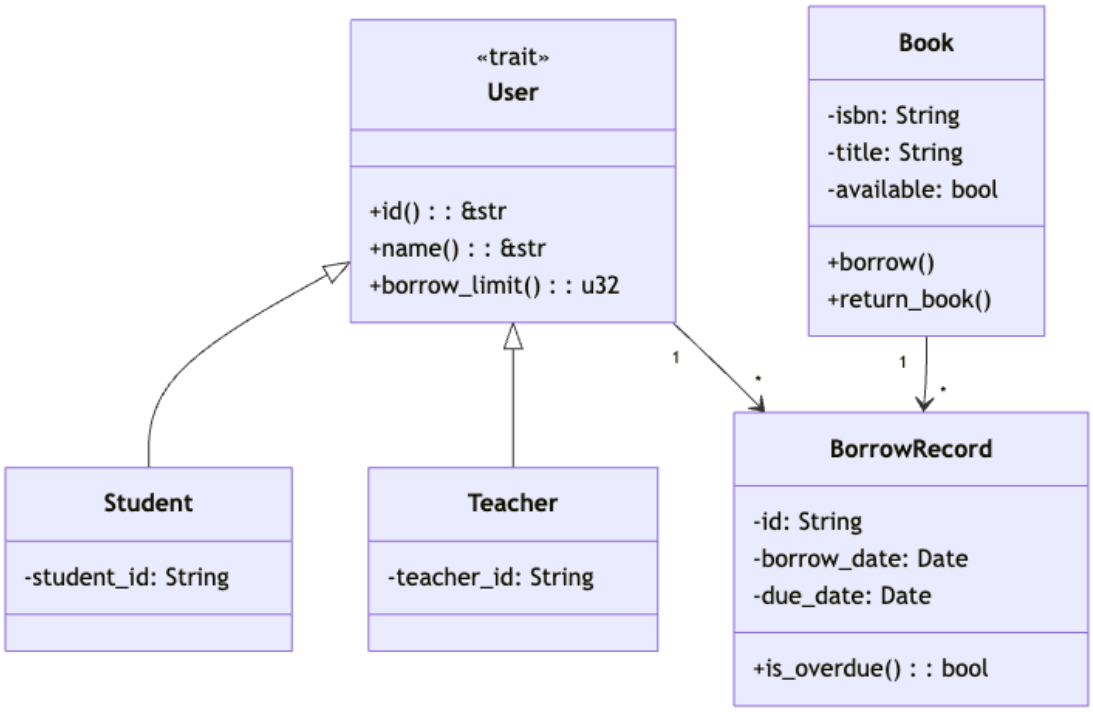
结论：UML不是孤立的，而是围绕一个功能从多个角度**完整刻画**系统。

从第3周成果出发

回顾类图

从第3周成果出发——回顾类图

第3周完成的类图（简化）



问题

有了类，但对象如何协作？业务流程如何？

这就是本周要补充的。

步骤1：用例图

定义系统边界与功能

步骤1——用例图：定义系统边界与功能

为什么先用用例图?

- 明确**谁**（参与者）使用系统的**哪些功能**（用例）
- 建立用户与系统的**交互契约**
- 指导后续动态建模的**范围**

借阅系统的用例图分析

参与者：学生、教师、图书管理员

核心用例：借书、还书、查询图书、管理图书、计算罚款

关系： – 借书和还书都包含”查询图书” – 还书包含”计算罚款” –“预约图书”扩展”借书”

AI生成用例图的提示词（课堂演示）

```
请为"高校图书借阅系统"生成用例图，使用Mermaid语法。

参与者：学生、教师、图书管理员
用例：登录、查询图书、借书、还书、管理图书、预约图书、查看借阅历史、计算罚款

关系：
- 学生和教师可以登录、查询图书、借书、还书、预约图书、查看借阅历史
- 图书管理员可以登录、管理图书、查看所有借阅记录
- 借书和还书都包含查询图书（<<include>>）
- 还书包含计算罚款（<<include>>）
- 预约图书扩展借书（<<extend>>）
```

> **讲解要点**：为什么这样定义参与者？为什么用include/extend？体现用例的**重用与扩展**。

步骤2：顺序图

细化”借书”用例的交互

步骤2——顺序图：细化”借书”用例的交互

为什么用顺序图?

- 用例图只说”借书”，但**如何借**？对象如何交互？
- 顺序图揭示对象间的**消息传递、时间顺序、激活周期**
- 帮助识别**缺失的对象或职责**

借书流程的交互分析

参与者：学生、UI、BorrowController、BookService、UserService、BorrowService、BookDB、BorrowDB

关键步骤：验证图书 → 验证用户借阅上限 → 创建记录 → 更新状态

异常处理：图书不存在、用户超限等用 `alt` 片段

AI生成顺序图的提示词（课堂演示）

```
请为"高校图书借阅系统"的借书流程生成顺序图，使用PlantUML语法。

参与者：学生、UI、BorrowController、BookService、UserService、BorrowService、BookDB、BorrowDB

流程：
1. 学生输入图书ID提交借书请求
2. UI调用控制器的borrowBook(bookId, userId)
3. 控制器调用BookService的getBook(bookId)
4. BookService查询BookDB返回图书信息
5. 控制器调用UserService的checkBorrowLimit(userId)
6. 控制器调用BorrowService的createBorrowRecord(userId, bookId)
7. BorrowService插入记录
8. 控制器调用BookService的updateBookStatus(bookId, 'borrowed')
9. 控制器返回成功
10. UI显示成功

使用alt片段处理图书不可借、用户超限等情况。
```

> **讲解要点**：激活条的作用？为什么需要控制器？alt片段如何对应if-else?

步骤3：状态图

刻画”图书”对象的生命周期

步骤3——状态图：刻画”图书”对象的生命周期

为什么用状态图?

- 类图只显示了图书的属性（如 `available: bool`），但无法表达状态转换的**规则和事件**
- 状态图回答：一个对象在什么事件下从什么状态变为什么状态，以及状态中的**内部活动**

图书状态分析

要素	内容
状态	可借、已借出、预约中、遗失、报废
事件	借书、还书、预约、取消预约、报失、找到、报废
内部活动	entry（进入时执行）、do（持续执行）、exit（退出时执行）

AI生成状态图的提示词（课堂演示）

```
请为"图书"对象生成状态图，使用PlantUML语法。

状态：可借（Available）、已借出（Borrowed）、预约中（Reserved）、遗失（Lost）、报废（Discarded）

初始状态：可借
终止状态：报废

转换事件：
- 借书：Available → Borrowed
- 预约：Available → Reserved
- 取消预约：Reserved → Available
- 借出预约图书：Reserved → Borrowed
- 还书：Borrowed → Available
- 报失：Borrowed → Lost
- 找到：Lost → Available
- 报废：Available/Borrowed/Lost → Discarded

在Available状态添加entry动作：打印"图书可借"
在Borrowed状态添加do动作：计时借阅天数
```

> **讲解要点**：为什么需要”预约中”状态？entry/do/exit分别对应什么时机？状态图如何指导代码中的状态模式或枚举?

步骤4：活动图

描述”还书”业务流程

步骤4——活动图：描述”还书”业务流程

为什么用活动图?

- 顺序图强调对象**交互**，活动图强调**流程和并发**
- 活动图可以跨越多个对象或**泳道**，展示业务过程中的**决策**、**并行**、**合并**

还书流程分析

要素	内容
泳道	学生、系统、仓库（体现职责分离）
决策	是否超期 → 计算罚款
并行	记录还书完成 与 仓库接收图书 可同时进行

AI生成活动图的提示词（课堂演示）

请为"高校图书借阅系统"的还书流程生成活动图，使用PlantUML语法。

泳道：学生、系统、仓库

流程：
- 学生提交还书请求
- 系统验证借阅记录
- 如果超期，计算罚款并生成罚单
- 系统更新图书状态为可借
- 并行：
  - 系统记录还书完成
  - 仓库接收图书（如果需要）
- 系统通知学生还书成功（并提示罚款金额）

使用决策节点处理超期情况。

> **讲解要点**：泳道对应什么？（对象或组织单元）并行如何实现？活动图与顺序图**互补关系**。

步骤5：组件图

模块化与可替换性

步骤5——组件图：模块化与可替换性

为什么用组件图?

- 类图描述**微观对象**，组件图描述**宏观模块**
- 组件是**物理封装**、**可替换**的单元，有明确的接口
- 组件图体现**高内聚**、**低耦合**的设计思想

借阅系统的组件划分

层次	组件
前端	用户交互
服务组件	借书、还书、查询、用户、罚款 → 每个服务有独立职责
数据访问层	统一的数据访问接口
数据库	持久化存储

AI生成组件图的提示词（课堂演示）

```
请为"高校图书借阅系统"生成组件图，使用PlantUML语法。

组件：
- 前端应用（Frontend）
- 借书服务（BorrowService）
- 还书服务（ReturnService）
- 查询服务（QueryService）
- 用户服务（UserService）
- 罚款服务（FineService）
- 数据访问层（DataAccess）
- 数据库（Database）

关系：
- 前端依赖所有服务
- 服务依赖数据访问层
- 数据访问层依赖数据库
- 借书服务和还书服务依赖用户服务和罚款服务

标注接口（提供的和需要的）。
```

> **讲解要点**：依赖倒置原则在组件图中的体现？接口如何降低耦合？

步骤6：部署图

物理架构与扩展性

步骤6——部署图：物理架构与扩展性

为什么用部署图?

- 组件图说明**逻辑部署**，部署图说明**物理部署**
- 明确软件运行在哪些硬件节点上，如何通信
- 为**性能**、**可用性**、**安全**等非功能需求提供基础

典型部署方案

节点	说明
客户端	浏览器/手机
Web服务器	Nginx（反向代理、负载均衡）
应用服务器	Tomcat（运行服务组件）
数据库	MySQL主从（读写分离）
缓存	Redis（加速查询）

AI生成部署图的提示词（课堂演示）

```
请为"高校图书借阅系统"生成部署图，使用PlantUML语法。

节点:
- 客户端设备（浏览器/手机）
- Web服务器（Nginx）
- 应用服务器（Tomcat）
- 数据库服务器（MySQL主库）
- 缓存服务器（Redis）
- 数据库从库

连接:
- 客户端通过HTTPS访问Web服务器
- Web服务器反向代理到应用服务器
- 应用服务器连接数据库主库和缓存
- 数据库主从复制
```

> **讲解要点**：为什么需要主从？缓存解决什么问题？部署图如何反映可扩展性？

高级内容：从OOD到OOP的迭代细化

高级内容——从OOD到OOP的迭代细化

为什么需要迭代?

- 一次性写出完美设计很难
- 通过迭代，逐步添加细节，验证可行性
- AI辅助可以快速将设计转化为代码，并提供反馈

迭代过程（四轮）

轮次	内容
第一轮	生成基础结构（trait、结构体、模块）
第二轮	完善业务逻辑（约束、计算、异常）
第三轮	增加持久化抽象（Repository模式，依赖倒置）
第四轮	生成可运行原型（CLI交互，内存存储）

核心思想

- **分离关注点**：业务逻辑与存储逻辑分开
- **依赖倒置**：服务依赖抽象接口，不依赖具体实现
- **开闭原则**：对扩展开放，对修改封闭

迭代细化示例——第一轮

基础结构

迭代细化示例——第一轮：基础结构

提示词

```
请为"高校图书借阅系统"生成Rust代码的基础结构，包括：
- models/user.rs：User trait, Student, Teacher
- models/book.rs：Book结构体及其方法
- models/borrow_record.rs：BorrowRecord结构体及其方法
- services/borrow_service.rs：BorrowService结构体，包含borrow_book方法
```

- main.rs：演示简单的借书流程

使用Rust 2021 edition。

目的

- 获得**可编译的骨架**
- 验证模块划分与依赖是否合理

迭代细化示例——第二轮

业务逻辑

迭代细化示例——第二轮：业务逻辑

提示词

在上一轮代码基础上，添加：
1. 在BorrowService中增加用户借阅数量检查（学生最多5本，教师最多10本）
2. 在BorrowRecord中实现is_overdue方法
3. 增加FineService，提供calculate_fine方法（每天0.5元）
4. 在main.rs中增加测试用例

目的

- 完善**核心算法**
- 添加必要的**业务规则**

迭代细化示例——第三轮

持久化抽象

迭代细化示例——第三轮：持久化抽象

提示词

请为系统增加持久化抽象：
- 定义BookRepository trait（find_by_isbn, update）
- 定义BorrowRepository trait（save, find_by_user）
- 创建内存实现（InMemoryBookRepository, InMemoryBorrowRepository）
- 修改BorrowService，使其依赖这些trait（依赖倒置）
- 更新main.rs，使用内存仓库演示完整流程

目的

- 遵循**SOLID原则**
- 使业务逻辑与存储**解耦**，便于测试和替换

迭代细化示例——第四轮

可运行原型

迭代细化示例——第四轮：可运行原型

提示词

请整合以上所有代码，生成一个完整的、可编译运行的Rust项目。

要求：
- 提供Cargo.toml（无需外部依赖）
- 模块化组织src/目录
- 在main.rs中实现交互式控制台程序，功能：
  1. 添加图书
  2. 注册学生/教师
  3. 借书
  4. 还书并计算罚款
  5. 查询图书状态
- 使用内存存储（Vec/HashMap）
- 包含错误处理，用户输入无效时给出提示

目的

- 获得可以**实际运行和演示**的系统
- 验证设计的**完整性与可行性**

从UML到代码的映射

从UML到代码的映射

类图 → 结构体、trait、关系

类图元素	代码实现
类名	结构体名
属性	字段
方法	方法
关联	字段或方法参数
继承	trait实现

顺序图 → 方法调用链

顺序图元素	代码实现
消息	方法调用
激活条	函数调用栈
alt	if-else或match

状态图 → 状态模式或枚举

状态图元素	代码实现
状态	枚举变体
转换	方法中的状态变更逻辑
entry/do/exit	状态机模式中的动作

活动图 → 控制流结构

活动图元素	代码实现
决策	if-else
并行	多线程或异步
泳道	不同类/模块的职责划分

综合案例分析

从需求到代码的完整演进

综合案例分析——从需求到代码的完整演进

案例：借书功能

阶段	内容
用例图	明确”学生”可以”借书”
类图	定义Student、Book、BorrowRecord、BorrowService
顺序图	细化交互步骤，发现需要UserService、BookService等
状态图	确定Book的状态转换规则
活动图	描述借书流程中的决策与并发（可选）
组件图	将借书功能封装为组件，明确依赖
部署图	决定借书服务部署位置
迭代细化	用AI生成代码，逐步实现

> 每个图都回答了不同问题，**共同保证设计正确性**

实验报告与评分

实验报告与评分

报告内容

- 各类UML图截图及说明（用例图、顺序图、状态图、活动图、组件图、部署图）
- 迭代细化过程记录（每一步的AI提示词和生成结果摘要）
- 最终原型系统的主要代码片段（关键部分）
- 运行演示截图
- AI实践心得（比较不同图生成的难易程度，AI在代码生成中的优缺点）

评分标准

检查项	分值
用例图正确完整	10分
顺序图正确完整	15分
状态图正确完整	10分

检查项	分值
活动图正确完整	15分
组件图正确合理	10分
部署图正确合理	10分
迭代细化过程记录	10分
可运行原型系统	15分
Git提交规范	5分

思考题

课堂讨论

思考题（课堂讨论）

1. 顺序图和活动图在描述流程时有什么区别？各适合什么场景？
2. 状态图是否适用于所有对象？什么情况下需要为对象建状态图？
3. 组件图和部署图在项目不同阶段的作用是什么？
4. 在迭代细化过程中，你遇到了哪些AI生成代码的问题？如何解决的？
5. 如果要将内存存储改为数据库存储，需要修改哪些部分？体现了什么设计原则？
6. 如何验证UML图与实际代码的一致性？

总结

总结

核心收获

- UML图是**多视角**的沟通工具，不是负担
- 从OOA到OOD到OOP是**逐步细化**的过程
- 动态建模（顺序图、状态图、活动图）补充了静态结构图（类图）的不足
- 架构图（组件图、部署图）关注**宏观组织与物理部署**

- AI可以帮助我们快速生成设计图和代码，但我们需要理解**为什么这样设计**

课后延伸

- 尝试为系统增加新的功能（如预约、邮件通知），看如何修改UML图和代码
- 将内存存储替换为SQLite或MySQL，体验Repository模式的好处
- 探索更多设计模式在系统中的运用

实验完成日期：_____

得分：_____

助教签名：_____